foundry10: R Package Reference Guide

Created March 2026 by Kara Weisman (kara@foundry10.org) with assistance from ClaudeAI
Maintained and distributed via GitHub: https://github.com/kgweisman/foundry10
Reference guide last updated: 2026-04-06

1. Setup

Install the foundry10 package directly from Github using the remotes package. (If needed, first install the remotes package by un-commenting the first line below.) The only strict dependency is ggplot2; the systemfonts package is optional but recommended for automatic Helvetica Neue font detection.

```
# install.packages("remotes")
remotes::install_github("kgweisman/foundry10")
```

Once installed, the package works like any other R package. You only need to install it once; if the package has been updated, you can install it again using the same code above to receive those updates.

To use the package, load it at the top of your script alongside any other packages you are using, for example:

```
library(foundry10)
library(tidyverse)
library(lme4)
```

2. Theme

`theme_foundry10()`

The main ggplot2 theme. Add it to any plot to apply the foundry10 visual identity: Helvetica Neue typography (falling back to Arial if Helvetica Neue is not available, and to the system default sans if Arial is not available), brand color accents, generous whitespace, and layout choices per the foundry10 Visual Style Guide.

Basic usage:

```
ggplot(...) +
  theme_foundry10() +
  labs(...)
```

Any element can be overridden by adding a `theme()` call afterward:

```
ggplot(...) +
  theme_foundry10() +
  theme(legend.position = "top",
        panel.grid.major.x = element_line(color = "pink", linewidth = 0.35))
```

Argument	Default	Description
`base_size`	`12`	Base font size in pt. Title is 1.75x, subtitle 1.2x, caption 0.8x, axis text 0.85x. Use 14–16 for slides, 10–11 for dense or faceted charts.
`base_family`	`.foundry10_font()`	Font family. Auto-detects Helvetica Neue if installed, then Arial, then the system sans-serif fallback.
`grid`	`TRUE`	TRUE shows subtle horizontal grid lines. FALSE removes all grid lines.
`strip_fill`	`"white"`	Facet strip background: `"white"`, `"orange"`, `"cyan"`, or `"black"`. Strip text color (white or black) is set automatically for contrast.

`preview_theme_foundry10()`

Renders a representative faceted scatter plot using the built-in `mpg` dataset, showcasing the full theme: title, subtitle, caption, legend, facet strips, grid lines, axis labels, and the discrete color palette. Useful for checking argument combinations before applying them to real charts.

Argument	Default	Description
`base_size`	`12`	Passed to `theme_foundry10()`.
`grid`	`TRUE`	Passed to `theme_foundry10()`.
`strip_fill`	`"white"`	Passed to `theme_foundry10()`.

3. Color scales

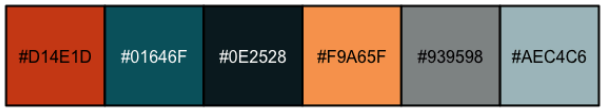
All scale functions come in fill and color variants (e.g. `scale_fill_foundry10_d()` and `scale_color_foundry10_d()`), plus `scale_colour_` aliases. Use fill variants for bars, tiles, areas, densities, etc.; use color variants for points, lines, etc.

`scale_fill_foundry10_d()` / `scale_color_foundry10_d()`

The `_d` ("discrete") variants of the `scale_fill_` / `scale_color_` functions provide a qualitative palette useful for categorical data where no color carries a positive/negative meaning, with up to 6 levels. This palette is colorblind-safe (tested against deuteranopia, protanopia, and tritanopia simulations).

Slots in palette order are accessible via the internal object `.palette_qualitative`:

```
> .palette_qualitative
[1] "#D14E1D" "#01646F" "#0E2528" "#F9A65F" "#939598" "#AEC4C6"
>
scales::show_col(.palette_qualitative, cex_label = 0.5, ncol = 6)
```



Usage note: Per the foundry10 Visual Style Guide, avoid assigning slot 1 (orange) to a positive or favorable category. Try reordering factor levels if needed; if your data has directional meaning, use a diverging scale instead.

Argument	Default	Description
`...`		Additional arguments passed to `ggplot2::discrete_scale()`.

`scale_fill_foundry10_seq()` / `scale_color_foundry10_seq()`

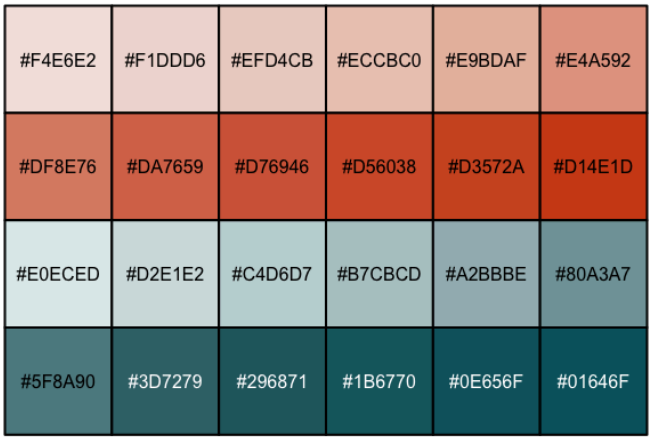
The `_seq` ("sequential") variants of the `scale_fill_` / `scale_color_` functions provide unipolar continuous palettes useful for continuous variables that run in one direction. Two palettes are available: orange (default) and cyan. These are smooth continuous ramps interpolated between four documented brand tints. By default, light tints represent low values, and dark tints represent high values.

Usage note: Per the foundry10 Visual Style Guide, consider using `tint = "orange"` when high values are negative or concerning, and using `tint = "cyan"` when high values are positive or favorable.

Argument	Default	Description
`tint`	`"orange"`	Color family: `"orange"` or `"cyan"`.
`direction`	`1`	`1` = light to dark (default). `-1` = dark to light.
`...`		Additional arguments passed to `ggplot2::scale_fill_gradientn()`.

These functions are supported by the low-level palette function `foundry10_pal_seq()`, which returns a character vector of n HEX codes interpolated across the selected brand tint ramp. Useful when you need raw HEX values for non-ggplot2 outputs, for example:

```
> foundry10_pal_seq(12)
 [1] "#F4E6E2" "#F1DDD6" "#EFD4CB" "#ECCBC0" "#E9BDAF" "#E4A592"
 [7] "#DF8E76" "#DA7659" "#D76946" "#D56038" "#D3572A" "#D14E1D"
>
> foundry10_pal_seq(12, tint = "cyan")
 [1] "#E0ECED" "#D2E1E2" "#C4D6D7" "#B7CBCD" "#A2BBBE" "#80A3A7"
 [7] "#5F8A90" "#3D7279" "#296871" "#1B6770" "#0E656F" "#01646F"
>
scales::show_col(c(foundry10_pal_seq(12), foundry10_pal_seq(12, tint = "cyan")),
                 cex_label = 0.5, ncol = 6)
```



Argument	Default	Description
`n`		Number of colors to return (no upper limit).
`tint`	`"orange"`	Color family: `"orange"` or `"cyan"`.
`direction`	`1`	`1` = light to dark (default). `-1` = dark to light.

`scale_fill_foundry10_seq_d()` / `scale_color_foundry10_seq_d()`

The `_seq_d` ("sequential discrete") variants of the `scale_fill_` / `scale_color_` functions provide a sequential palette useful for ordered categorical data where color carries the meaning of intensity, up to 4 levels (e.g., low / medium / high / very high; never / rarely / sometimes / often). Factor levels should run from least to most (or most to least if `direction = -1`).

Usage note: Per the foundry10 Visual Style Guide, consider using `tint = "orange"` when high values are negative or concerning, and using `tint = "cyan"` when high values are positive or favorable.

Argument	Default	Description
`n`		Number of levels (2 through 4).
`tint`	`"orange"`	Color family: `"orange"` or `"cyan"`.
`direction`	`1`	`1` = light to dark (default). `-1` = dark to light.
`...`		Additional arguments passed to `ggplot2::scale_fill_manual()`.

These functions are supported by the low-level palette function `foundry10_pal_seq_d()`, which returns a character vector of n fixed brand tint HEX codes. Useful when you need raw HEX values for non-ggplot2 outputs, for example:

```
> foundry10_pal_seq_d(4) # change to values <= 4 as desired
[1] "#F4E6E2" "#EBC5B9" "#D96F50" "#D14E1D"
>
> foundry10_pal_seq_d(4, tint = "cyan") # change to values <= 4 as desired
[1] "#E0ECED" "#AEC4C6" "#326A72" "#01646F"
>
> scales::show_col(c(foundry10_pal_seq_d(4), foundry10_pal_seq_d(4, tint = "cyan")),
                   cex_label = 0.5, ncol = 4)
```

Argument	Default	Description
`n`		Number of colors to return (2 through 4).
`tint`	`"orange"`	Color family: `"orange"` or `"cyan"`.
`direction`	`1`	`1` = light to dark (default). `-1` = dark to light.

`scale_fill_foundry10_div()` / `scale_color_foundry10_div()`

The `_div` ("diverging") variants of the `scale_fill_` / `scale_color_` functions provide a bipolar continuous palette useful for continuous variables with a meaningful midpoint (e.g., correlations running from -1 to +1). These are smooth continuous ramps interpolated between three documented brand colors: orange (low) → light gray (midpoint) → cyan (high). Set `limits = c(-x, x)` to anchor the gray midpoint precisely at zero.

Usage note: Per the foundry10 Visual Style Guide, consider plotting negative or concerning values in orange and positive or favorable values in cyan.

Argument	Default	Description
`direction`	`1`	`1` = orange-to-cyan (default). `-1` = reversed.
`...`		Additional arguments passed to `ggplot2::scale_fill_gradientn()`. Use `limits = c(-x, x)` to center the midpoint.

These functions are supported by the low-level palette function `foundry10_pal_div()`, which returns a character vector of n HEX codes interpolated across the orange → gray → cyan diverging ramp. Useful when you need raw HEX values for non-ggplot2 outputs, for example:

```
> foundry10_pal_div(24)
 [1] "#D14E1D" "#D25A2E" "#D3673F" "#D57450" "#D68162" "#D88E73"
 [7] "#D99B84" "#DBA896" "#DCB5A7" "#DEC2B8" "#DFCFCA" "#E1DCDB"
[13] "#D8DDDE" "#C4D2D4" "#B1C7CA" "#9DBCC0" "#89B1B6" "#76A6AC"
[19] "#629BA1" "#4F9097" "#3B858D" "#287A83" "#146F79" "#01646F"
>
scales::show_col(foundry10_pal_div(24), cex_label = 0.5, ncol = 6)
```

Argument	Default	Description
`n`		Number of colors to return.
`direction`	`1`	`1` = orange-to-cyan (default). `-1` = reversed.

`scale_fill_foundry10_div_d()` / `scale_color_foundry10_div_d()`

The `_div_d` ("diverging discrete") variants of the `scale_fill_` / `scale_color_` functions provide a bipolar discrete palette useful for ordered categorical variables with a meaningful midpoint, up to 9 response levels (e.g., Likert-scale survey responses running from "Definitely no" to "Definitely yes"). Odd values of n get a gray neutral midpoint; even values split symmetrically between orange and cyan sides. Factor levels should run from negative to positive (or positive to negative if `direction = -1`).

Usage note: Per the foundry10 Visual Style Guide, consider plotting negative or concerning values in orange and positive or favorable values in cyan.

Argument	Default	Description
`n`		Number of response levels (2 through 9).
`direction`	`1`	`1` = orange-to-cyan (default). `-1` = reversed.

`...`		Additional arguments passed to `ggplot2::scale_fill_manual()`.

These functions are supported by the low-level palette function `foundry10_pal_div_d()`, which returns a character vector of n fixed brand color HEX codes ordered from most negative (orange) through neutral (gray, odd n only) to most positive (cyan). Useful when you need raw HEX values for non-ggplot2 outputs, for example:

```
> foundry10_pal_div_d(9) # change to values <= 9 as desired
[1] "#D14E1D" "#D96F50" "#EBC5B9" "#F4E6E2" "#E2E3E4" "#E0ECED"
[7] "#AEC4C6" "#326A72" "#01646F"
>
> scales::show_col(foundry10_pal_div_d(9), cex_label = 0.4, ncol = 9)
```

Argument	Default	Description
`n`		Number of colors to return (2 through 9).
`direction`	`1`	`1` = orange-to-cyan (default). `-1` = reversed.

4. Color utilities

`foundry10_colors`

A named list containing all official foundry10 brand colors, organized into five palette groups. This is a data object, not a function; you can access it directly or use the helper functions below.

`primary`

	Name	HEX
	`orange_foundry10`	`#D14E1D`
	`cyan_foundry10`	`#01646F`
	`black`	`#111111`
	`offwhite`	`#F9F9F9`

`secondary`

	Name	HEX
	`cyan_warm`	`#077A8A`
	`cyan_cool`	`#007079`

	`cyan_bright`	`#078B9C`
	`cyan_verydark`	`#0E2528`
	`orange_light`	`#F9A65F`
	`orange_soft`	`#E77C53`
	`orange_text`	`#CA4A1C`
	`orange_websafebackground`	`#BE5127`

`tints_orange`

	Name	HEX
	100% (primary)	`#D14E1D`
	80% tint	`#D96F50`
	20% tint	`#EBC5B9`
	10% tint	`#F4E6E2`

Note: `tints_orange` is stored as an unnamed vector in `foundry10_colors`. Color names above are descriptive labels, not R object names.

`tints_cyan`

	Name	HEX
	100% (primary)	`#01646F`
	80% tint	`#326A72`
	20% tint	`#AEC4C6`
	10% tint	`#E0ECED`

Note: `tints_cyan` is stored as an unnamed vector in `foundry10_colors`. Color names above are descriptive labels, not R object names.

`gray`

	Name	HEX
	`black_absolute`	`#000000`
	`gray_dark`	`#575857`
	`gray_mediumdark`	`#939598`
	`gray_light`	`#E2E3E4`
	`gray_verylight`	`#F2F2F2`

	`white_absolute`	`#FFFFFF`

`foundry10_color()`

Looks up one or more colors by name and returns their HEX codes. Accepts any color name from `foundry10_colors`.

Example: `foundry10_color("orange_foundry10")` returns `"#D14E1D"`.

Argument	Default	Description
`name`		One or more color name strings, e.g. `"orange_foundry10"`, `"cyan_cool"`, `"gray_light"`. See `foundry10_colors`.

`foundry10_palette()`

Returns a named character vector of HEX codes for an entire palette group. Useful for extracting colors programmatically or passing them to other plotting functions.

Example: `foundry10_palette("primary")` returns a named vector of 4 HEX codes:

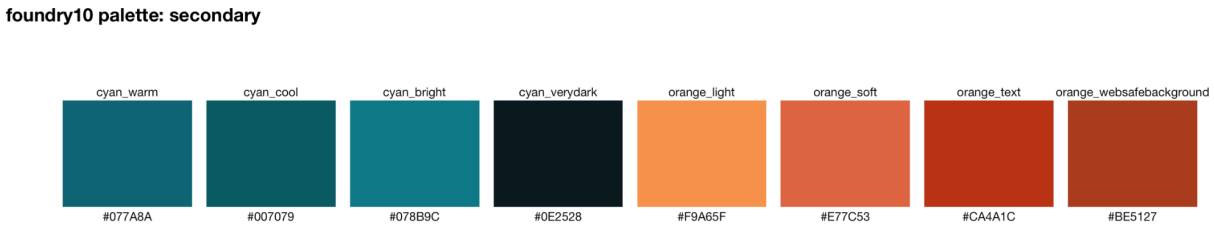
```
orange_foundry10   cyan_foundry10            black         offwhite
       "#D14E1D"        "#01646F"        "#111111"        "#F9F9F9"
```

Argument	Default	Description
`palette`	`"primary"`	One of: `"primary"`, `"secondary"`, `"tints_orange"`, `"tints_cyan"`, `"gray"`, or `"all"`. See `foundry10_colors`.

`preview_foundry10_palette()`

Renders a ggplot2 color swatch showing all colors in the selected palette group, labeled with color names and HEX codes. Not particularly useful for dataviz, but perhaps useful for checking colors against the foundry10 Visual Style Guide more generally.

Example: `preview_foundry10_palette("secondary")` returns the following ggplot:



Argument	Default	Description
`palette`	`"primary"`	One of: `"primary"`, `"secondary"`, `"tints_orange"`, `"tints_cyan"`, `"gray"`, or `"all"`. See `foundry10_colors`.

5. Quick reference

Common patterns at a glance. All assume `library(foundry10)` and library(ggplot2) are loaded.

Chart type	Scale + theme

Categorical (unordered)	`ggplot(aes(color = var, ...), ...) +`
	`  scale_color_foundry10_d() +`
	`  theme_foundry10()`

Heatmap / tile (orange = more/worse)	`ggplot(aes(fill = var, ...), ...) +`
	`  scale_fill_foundry10_seq() +`
	`  theme_foundry10(grid = FALSE)`

Heatmap / tile (cyan = more/better)	`ggplot(aes(fill = var, ...), ...) +`
	`  scale_fill_foundry10_seq(tint = "cyan") +`
	`  theme_foundry10(grid = FALSE)`

Above/below baseline, continuous, running from -1 to +1	`ggplot(aes(fill = var, ...), ...) +`
	`  scale_fill_foundry10_div(limits = c(-1, 1)) +`
	`  theme_foundry10()`

Likert / ordered with midpoint (5-point)	`ggplot(aes(fill = var, ...), ...) +`
	`  scale_fill_foundry10_div_d(n = 5) +`
	`  theme_foundry10()`

Ordered categories (4-level)	`ggplot(aes(fill = var, ...), ...) +`
	`  scale_fill_foundry10_seq_d(n = 4) +`
	`  theme_foundry10()`

Faceted chart with orange strips	`ggplot(aes(color = var, ...), ...) +`
	`  facet_grid(...) +`
	`  scale_color_foundry10_d() +`
	`  theme_foundry10(strip_fill = "orange")`

Slides (larger text)	`ggplot(...) +` `  theme_foundry10(base_size = 16)`
No grid lines	`ggplot(...) +` `  theme_foundry10(grid = FALSE)`

6. Function index

Function	Purpose
`theme_foundry10()`	Main ggplot2 theme

`preview_theme_foundry10()`	Render a sample plot showing the full theme
`scale_fill_foundry10_d()`	Qualitative fill scale (up to 6 categorical levels)
`scale_color_foundry10_d()`	Qualitative color scale (up to 6 categorical levels)
`scale_fill_foundry10_seq()`	Continuous sequential fill scale (orange or cyan ramp)
`scale_color_foundry10_seq()`	Continuous sequential color scale (orange or cyan ramp)
`foundry10_pal_seq()`	Low-level: return n HEX codes from sequential ramp
`scale_fill_foundry10_seq_d()`	Discrete sequential fill scale (2–4 ordered levels)
`scale_color_foundry10_seq_d()`	Discrete sequential color scale (2–4 ordered levels)
`foundry10_pal_seq_d()`	Low-level: return n fixed tint HEX codes (sequential)
`scale_fill_foundry10_div()`	Continuous diverging fill scale (orange → gray → cyan)
`scale_color_foundry10_div()`	Continuous diverging color scale (orange → gray → cyan)
`foundry10_pal_div()`	Low-level: return n HEX codes from diverging ramp
`scale_fill_foundry10_div_d()`	Discrete diverging fill scale (2–9 Likert levels)
`scale_color_foundry10_div_d()`	Discrete diverging color scale (2–9 Likert levels)
`foundry10_pal_div_d()`	Low-level: return n fixed brand color HEX codes (diverging)
`foundry10_colors`	Data object: named list of all brand colors by palette group
`foundry10_color()`	Look up a single color by name, return HEX code
`foundry10_palette()`	Return a named HEX vector for a palette group
`preview_foundry10_palette()`	Render a color swatch for a palette group

Note: All `scale_fill_` functions have matching `scale_color_` / `scale_colour_` aliases not listed above. The `scale_colour_` aliases (British spelling) are identical to `scale_color_` variants.